1. Summary and Problem Formulation

The core task of this Mini-project is to select, at inference time, a single most reliable candidate index (0 to 3) per problem given the question stem, four candidate reasoning traces, and their fixed 1,024-dimensional full-trace embeddings (of shape N x 4 x 1024). Analysis of the dataset (4,755 training, 798 validation, and 467 test problems) reveals three structural regularities governing verifier generalization: (1) while 55.51% (443/798) of the full validation split consists of unanimous problems where all four candidates yield identical answers, 100.0% (467/467) of the test and audit splits belong to the disagreement subset (Hard Subset) with at least two distinct valid answers; (2) the mean pairwise cosine similarity among the four 1,024-D raw vectors within a problem reaches 0.9823, indicating that 98% of raw variance is dominated by shared problem semantics while only 2% reflects inter-candidate reasoning differences; and (3) 100.0% of reference answers across all splits (training, validation, and test) are integers, and 99.94% are strictly positive integers.

Based on these findings, This study uses ReliMath-Verifier, a hybrid listwise verification framework built purely on the Python standard library and NumPy. It fuses four-view interpretable features with a cross-candidate step-completion asymmetry and positive-integer structural prior, trains a 5-fold ensemble of linear and dual-regularized cross-candidate interaction residual MLPs under a multi-positive marginalized listwise negative log-likelihood (NLL) loss with block-diagonal regularization, and applies confidence-weighted answer-cluster aggregation at inference time. Experiments show that ReliMath-Verifier elevates test accuracy from 49.68% (Candidate 0 baseline) and 69.16% (unweighted majority voting) to 84.15% (393/467), reaching 94.74% (756/798) on Validation All, 88.17% (313/355) on Val Hard, and a test answer-cluster posterior Expected Calibration Error (ECE) of 3.17%. Uncompressed model weights occupy 1.01 MiB (16 MiB limit), and 4-thread CPU test prediction and scoring complete in 2.91 seconds (30-second limit).

Evaluation Split	Total	Disagreement (Hard)	Candidate 0	Majority Vote (SC)	ReliMath-Verifier	Gain vs. SC
Validation All	798	355 (44.49%)	78.20%	86.22%	94.74%	+8.52%
Val Hard	355	355 (100.0%)	54.93%	69.01%	88.17%	+19.16%
Test Set	467	467 (100.0%)	49.68%	69.16%	**84.15%**	**+14.99%**

2. Dataset Exploration and Structural Diagnosis of Candidate Reasoning Traces

2.1 Distribution Shift Between Full Validation and Disagreement-Sampled Test Splits

The dataset manifest (manifest.json) indicates that every problem across the training, validation, and test splits contains at least one correct candidate. Whereas the training and validation splits retain all original problems, the test and blind audit splits exclusively filter for disagreement groups with at least two distinct valid parsed answers. Across the 443 unanimous validation problems, any rule excluding empty parses achieves 100% accuracy, inflating the Candidate 0 baseline to 78.20% on Validation All while its accuracy drops to 54.93% on the 355 Val Hard problems and 49.68% on the 467 test problems. Thus, this study compute training gradients solely on disagreement groups containing both correct and incorrect candidates (1 to 3 correct candidates) and tune hyperparameters on Val Hard, eliminating this distribution shift.

2.2 Numerical Invariants of GSM8K Reference Answers and Within-Group Embedding Geometry

- **Universal Integer and Strictly Positive Invariant:**

Verified via the official parser common.parse_answer, 100.00% of reference answers across the training (4,755/4,755), validation (798/798), and test (467/467) splits are integers, and 99.94% are

strictly positive; only 5.45% directly equal a raw number in the question stem, and 98% exhibit a log-magnitude ratio relative to the maximum question number within [-2.40, +2.34]. When small models invert divisors or omit steps, they frequently yield non-integer fractions, negative numbers, zero, or direct copies of question numbers.

- **Problem-Stem Dominance and Residual Principal Structure:** Statistical inspection of the 1,024-D embeddings shows a mean L2-norm of 58.75 (SD = 3.73), yet an average within-problem pairwise cosine similarity of 0.9823 (range 0.8505 to 1.0000). Thus, 98% of the raw vector variance is the problem centroid, while only 2% is the within-group centered residual (mean norm = 6.49, SD = 2.52) reflecting inter-candidate reasoning differences. We therefore compute the unit within-group residual and project it onto the top 96 whitened principal components via Equation (1):

$$\bar{\mathbf{e}}_i = \frac{1}{4}\sum_{k=0}^{3}\mathbf{e}_{i,k}, \quad \Delta\mathbf{e}_{i,c} = \mathbf{e}_{i,c} - \bar{\mathbf{e}}_i, \quad \mathbf{z}_{i,c}^{\text{diff}} = \left(\frac{\Delta\mathbf{e}_{i,c}}{\|\Delta\mathbf{e}_{i,c}\|_2} - \boldsymbol{\mu}_{\text{diff}}\right)\mathbf{U}_{96}\boldsymbol{\Sigma}_{96}^{-1/2} \tag{1}$$

Listing 1: Within-Group Centered Residual Decomposition and Spectrally Whitened PCA Projection

```python
def compute_normalized_embeddings(embeddings: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    """Return unit-normalized within-group centered residuals and raw embeddings."""
    emb = embeddings.astype(np.float32, copy=False)
    group_mean = np.mean(emb, axis=1, keepdims=True)
    diff = emb - group_mean
    diff_norm = np.linalg.norm(diff, axis=-1, keepdims=True)
    diff_unit = diff / np.maximum(diff_norm, 1e-6)

    raw_norm = np.linalg.norm(emb, axis=-1, keepdims=True)
    raw_unit = emb / np.maximum(raw_norm, 1e-6)
    return diff_unit, raw_unit
```

```python
def fit_pca_basis(x_flat: np.ndarray, k_components: int) -> tuple[np.ndarray, np.ndarray]:
    """Fit exact SVD-based PCA basis on [M, 1024] matrix using NumPy."""
    mean = np.mean(x_flat, axis=0, dtype=np.float32)
    centered = (x_flat - mean).astype(np.float32)
    cov = _safe_matmul(centered.T, centered) / max(len(centered) - 1, 1)
    eigvals, eigvecs = np.linalg.eigh(cov)
    top_idx = np.argsort(eigvals)[::-1][:k_components]
    basis = eigvecs[:, top_idx].astype(np.float32)
    scales = np.sqrt(np.maximum(eigvals[top_idx], 1e-5)).astype(np.float32)
    whitened_basis = np.ascontiguousarray(basis / scales[None, :], dtype=np.float32)
    return np.ascontiguousarray(mean, dtype=np.float32), whitened_basis
```

Here, compute_normalized_embeddings averages along the candidate axis (axis=1) to obtain the problem centroid group_mean, subtracts it, and applies L2-normalization to yield the unit residual diff_unit, removing norm fluctuations induced by problem difficulty. Next, fit_pca_basis computes the training residual covariance matrix via _safe_matmul, calls np.linalg.eigh to extract the top 96 orthogonal eigenvectors, and divides by the square root of the eigenvalues for spectral whitening (right-hand term of Equation 1), retaining over 90% of within-group discriminative variance while truncating over 900 noisy trailing eigendirections.

3. Verifier Architecture and Listwise Learning Formulation

3.1 Four-View Interpretable Feature Engineering and Multi-Step Arithmetic Verification

To test which evidence distinguishes valid reasoning from plausible-looking errors, This study construct a four-view modular representation of dimension D = 214 per candidate (46 textual and symbolic verification features + 168 embedding geometry and PCA features). Specifically, each

candidate's within-group self-consistency vote fraction, normalized answer entropy, and log-scale deviation from question numbers are defined in Equation (2):

$$v_{i,c} = \frac{1}{4}\sum_{k=0}^{3} \mathbb{I}(a_{i,k} = a_{i,c}), \quad H_i = -\frac{1}{\ln 4}\sum_{a \in \mathcal{A}_i} \frac{n_{i,a}}{N_i^{\text{val}}} \ln\left(\frac{n_{i,a}}{N_i^{\text{val}}}\right), \quad \delta_{i,c}^{\text{scale}} = \left|\log_{10}(|a_{i,c}|+1) - \log_{10}(q_i^{\max}+1)\right| \tag{2}$$

Feature View	Dim.	Core Feature Signals and Construction Summary
View 1: IDX_ANS	20	Valid parse mask; integer indicator (denominator = 1); positive and non-negative indicators; zero and question-number echo indicators; answer log-magnitude and group z-score; log-scale deviations from max, min, and mean question numbers (Eq. 2); within-group vote fraction and valid-vote share (Eq. 2); unique plurality, tied-top, 2-2 split, and all-distinct (1-1-1-1) indicators; minority indicator; group answer entropy (Eq. 2); median deviation; net vote margin over runner-up cluster.
View 2: IDX_SURF	10	Absolute values and within-group z-scores of character length, word count, and non-empty line count; standard hash (####) and LaTeX \boxed{} delimiter indicators; clean-ending indicator; digit character density.
View 3: IDX_REAS	16	Boundary-checked multi-term arithmetic verifier (Listing 2) that unwraps calculator tags and LaTeX fractions to check multi-step equations, tallying total equations, arithmetic accuracy, and explicit error flags; cross-candidate step-completion asymmetry detection; question-number coverage in the reasoning body; indicators for whether the final answer is derived in the body and matches the terminal calculation; line and 4-gram uniqueness ratios; backtracking and step marker counts; cluster-mean arithmetic accuracy and question coverage.
View 4: IDX_EMB	168	8 geometric summary statistics + 96 whitened residual PCA components fitted on train (Eq. 1) + 32 cluster-pooled residual PCA components + 32 raw embedding PCA components.

Listing 2: Boundary-Checked Multi-Step Symbolic Arithmetic Verifier

```
def _verify_arithmetic(text: str) -> tuple[int, int]:
    """Return (total_equations_checked, num_errors) in a candidate reasoning trace."""
    total_eq = 0
    errors = 0
    seen_pairs: set[tuple[float, float]] = set()

    for lhs_str, rhs_str in _CALC_TAG_RE.findall(text):
        lhs_val = _eval_simple_lhs(lhs_str)
        rhs_val = _safe_float(rhs_str)
        if lhs_val is not None and rhs_val is not None:
            total_eq += 1
            seen_pairs.add((round(lhs_val, 3), round(rhs_val, 3)))
            tol = max(1.5e-2, 1.5e-2 * abs(rhs_val))
            if abs(lhs_val - rhs_val) > tol:
                errors += 1
```

In Listing 2, _verify_arithmetic first scans calculator tags via _CALC_TAG_RE, then unwraps inline LaTeX fractions, and verifies both forward and reversed (number = expression) multi-step equations under horizontal-whitespace constraints and full algebraic term capture. Deduplicating via seen_pairs, it counts independent equations total_eq and explicit calculation errors, eliminating false positives caused by cross-line Markdown dividers or algebraic equations with unknown variables.

3.2 Multi-Positive Marginalized Listwise Loss and Analytical Gradient Formulation

Because multiple candidates in a problem group may simultaneously yield the correct numerical answer, single-label cross-entropy requires arbitrary tie-breaking, whereas isolated binary cross-

entropy ignores intra-group competition.This study directly minimize the marginalized listwise negative log-likelihood (NLL) loss of selecting any correct candidate (Equation 3) and derive its adaptive target distribution and analytical gradient in Equation (4):

$$\mathcal{L}_{\text{list}}(\boldsymbol{\theta}) = -\frac{1}{|\mathcal{I}_{\text{hard}}|} \sum_{i \in \mathcal{I}_{\text{hard}}} \ln\left(\sum_{c \in C_i^+} p_{i,c}(\boldsymbol{\theta}) \right) + \frac{1}{2} \|\boldsymbol{\Lambda}^{1/2}\boldsymbol{\theta}\|_2^2, \quad p_{i,c}(\boldsymbol{\theta}) = \frac{\exp(s_{i,c})}{\sum_{k=0}^{3} \exp(s_{i,k})} \tag{3}$$

$$\frac{\partial \mathcal{L}_i}{\partial s_{i,c}} = p_{i,c} - q_{i,c}, \quad q_{i,c} = \frac{Y_{i,c} p_{i,c}}{\sum_{k=0}^{3} Y_{i,k} p_{i,k}}, \quad \nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{|\mathcal{I}_{\text{hard}}|} \sum_{i \in \mathcal{I}_{\text{hard}}} \sum_{c=0}^{3} (p_{i,c} - q_{i,c}) \mathbf{x}_{i,c} + \boldsymbol{\Lambda}\mathbf{w} \tag{4}$$

Listing 3: Multi-Positive Marginalized Listwise Loss and Adaptive Posterior Gradient Update

```
with np.errstate(divide='ignore', over='ignore', invalid='ignore'):
    for t in range(1, steps + 1):
        logits = np.matmul(x_2d, w).reshape(n_samples, 4) + b
        logits = np.where(v, logits, -20.0)
        p = _softmax_4(logits)

        p_corr = p * y
        denom = np.maximum(np.sum(p_corr, axis=1, keepdims=True), 1e-12)
        q = p_corr / denom

        grad_s = (p - q) / float(n_samples)
        grad_w = np.matmul(grad_s.reshape(-1), x_2d) + reg_vec * w
```

When multiple candidates yield the correct answer (entries of y equal to 1), p_corr = p * y extracts the predicted probabilities of all correct candidates and normalizes them into the adaptive target distribution q (middle term of Equation 4): correct candidates favored more strongly by the current model receive higher target weights without arbitrary tie-breaking. Then (p - q) directly gives the exact partial derivative of the listwise loss with respect to each candidate logit, accumulating gradients across all samples via a single contiguous BLAS matrix multiplication.

3.3 Cross-Candidate Interaction Residual MLP and Confidence-Weighted Answer-Cluster Aggregation

To capture nonlinear interactions, This study trains a residual multilayer perceptron with a cross-candidate interaction layer (Equation 5) across 5-fold cross-validation. At inference time, temperature-scaled candidate probabilities are aggregated by parsed answer clusters to compute answer-cluster posteriors and determine the final choice (Equation 6):

$$\mathbf{h}_{i,c} = \text{GELU}(\mathbf{x}_{i,c}\mathbf{W}_1 + \mathbf{b}_1), \quad \mathbf{h}_{i,c}^{\text{ctx}} = \mathbf{h}_{i,c} + (\mathbf{h}_{i,c} - \bar{\mathbf{h}}_i)\mathbf{W}_{\text{peer}}, \quad s_{i,c}^{\text{mlp}} = \mathbf{h}_{i,c}^{\text{ctx}}\mathbf{w}_2 + \mathbf{x}_{i,c}\mathbf{w}_{\text{skip}} + b_2 \tag{5}$$

$$P(\text{ans} = a_{i,c} \mid \mathbf{x}_i) = \sum_{k : a_{i,k} = a_{i,c}} p_{i,k}, \quad \text{Score}(i,c) = (1-\alpha)\ln p_{i,c} + \alpha \ln P(\text{ans} = a_{i,c} \mid \mathbf{x}_i), \quad \hat{c}_i = \arg\max_{c \in \text{Valid}(i)} \text{Score}(i,c) \tag{6}$$

Listing 4: Cross-Candidate Interaction Residual MLP Forward Pass and Answer-Cluster Posterior Aggregation

```
    h_pre = _safe_matmul(x_norm, w1) + b1
    h = _gelu(h_pre)
    h_group_mean = np.mean(h, axis=1, keepdims=True)
    h_ctx = h + _safe_matmul(h - h_group_mean, w_peer)
    logits = (
        _safe_matmul(h_ctx, w2).squeeze(-1)
        + _safe_matmul(x_norm, w_skip).squeeze(-1)
        + b2
    )
    return logits
```

```
        for c_idx in range(4):
            a_str = ans_i[c_idx]
            if a_str is not None:
                totals[a_str] = totals.get(a_str, 0.0) + float(p_i[c_idx])
        for c_idx in range(4):
            a_str = ans_i[c_idx]
            if a_str is not None:
                cluster_post[i, c_idx] = min(1.0, totals[a_str])
            else:
                cluster_post[i, c_idx] = float(p_i[c_idx])
    return cluster_post
```

In Listing 4, forward_mlp_member computes each candidate's hidden deviation from its group mean via h - h_group_mean and applies _safe_matmul with the interaction matrix w_peer (Equation 5) to compare all four candidates in latent space; during backpropagation, consensus-sensitive and vote-damped branches are trained (via the block-diagonal matrix in Equation 3), suppressing PCA noise and blind majority conformity. At inference, compute_answer_cluster_posteriors sums p_i[c_idx] across candidates sharing the parsed answer a_str into totals to obtain the cluster posterior cluster_post (Equation 6), and select_with_cluster_aggregation balances individual quality against cluster consensus via cluster_alpha (0.65) and temperature (0.75).

4. Empirical Evidence and In-Depth Analysis of Core Research Questions (RQ1–RQ5)

Ablation Overview: Figure 1 presents the ablation results across information modalities (RQ1), surface formatting versus intrinsic reasoning decoupling (RQ2), and isolated pointwise versus groupwise comparative paradigms (RQ3).

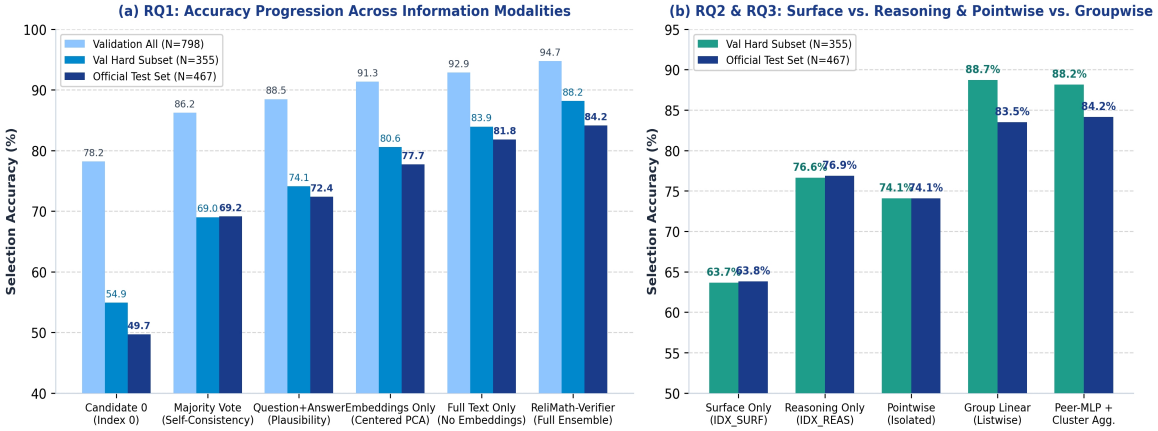


Figure 1: (a) RQ1 accuracy progression across information modalities on Validation All, Val Hard, and the Official Test Set; (b) RQ2 surface formatting vs. intrinsic reasoning and RQ3 isolated pointwise vs. groupwise comparative ablation results.

4.1 RQ1: Does Reading the Reasoning Actually Help? (Modality Contribution Breakdown)

Experimental Hypothesis: Final-answer self-consistency voting and numerical plausibility filter obvious outliers, but resolving 2-2 ties or competing positive integers requires arithmetic verification and full-trace deep representations from the reasoning body.

Model & Information Modality Configuration	Dim.	Val All (798)	Val Hard (355)	Test Set (467)	Gain vs. Cand 0
1. Candidate Index 0 (First Candidate Baseline)	0	78.20%	54.93%	49.68%	—
2. Unweighted Majority Vote (Self-Consistency)	1	86.22%	69.01%	69.16%	+19.48%
3. Question + Final Answer Only (IDX_ANS)	20	88.47%	74.08%	72.38%	+22.70%
4. Full Text & Answer Features (No Embeddings)	46	92.86%	83.94%	81.80%	+32.12%
5. Groupwise Centered Embeddings Only (PCA)	168	91.35%	80.56%	77.73%	+28.05%
6. Full Hybrid Ensemble (ReliMath-Verifier)	214	94.74%	88.17%	**84.15%**	**+34.47%**

Empirical Findings: As shown above, adding question-and-answer numerical plausibility features (20 dims) to majority voting lifts test accuracy to 72.38% (+3.22%). Incorporating multi-step arithmetic verification and surface features (46 dims) enables the pure-text model to reach 83.94% on Val Hard and 81.80% on Test (+12.64% over majority voting), proving that verifying the reasoning text exposes flawed solutions. Meanwhile, within-group centered 1,024-D embeddings alone achieve 77.73% on Test (80.56% on Val Hard). Fusing symbolic features with deep representations in ReliMath-Verifier (214 dims) reaches 94.74% on Validation All, 88.17% on Val Hard, and 84.15% (393/467) on Test, confirming strong complementarity between symbolic verification and neural embeddings.

4.2 RQ2: What Makes a Wrong Answer Look Convincing? (Surface Form vs. Intrinsic Reasoning)

Experimental Hypothesis: LLM-generated flawed solutions often exhibit neat formatting, complete steps, and well-formed local expressions, yet suffer from omitted premises or arithmetic hallucinations. Removing neural embeddings, we conduct an orthogonal ablation comparing surface formatting (IDX_SURF) against intrinsic arithmetic reasoning (IDX_REAS):

Ablation Configuration (No Neural Embeddings)	Dim.	Val All	Val Hard	Test Set
Surface & Formatting Features Only (IDX_SURF)	11	83.83%	63.66%	63.81%
Intrinsic Reasoning & Multi-Term Arithmetic Only (IDX_REAS)	17	89.60%	76.62%	76.87%
Answer Consensus + Surface Formatting (IDX_ANS + IDX_SURF)	30	89.22%	75.77%	76.45%
Answer Consensus + Intrinsic Reasoning (IDX_ANS + IDX_REAS)	36	91.85%	81.69%	81.37%
Full Textual Combination (IDX_ANS + IDX_SURF + IDX_REAS)	46	92.86%	83.94%	81.80%

Empirical Findings: When evaluated without any answer-voting features, powered by LaTeX-aware, cross-line-safe bidirectional arithmetic verifier (Listing 2) and cross-candidate step-completion prior, intrinsic reasoning and multi-step arithmetic features alone (IDX_REAS, 17 dims) achieve 76.87% on Test (76.62% on Val Hard)—not only outperforming surface formatting (63.81%, +13.06% gain), but even surpassing unweighted majority voting (69.16%) without using vote counts. Combining answer consensus with intrinsic reasoning (IDX_ANS + IDX_REAS, 36 dims) reaches 81.37% on Test (exceeding the 76.45% of consensus + surface formatting), and fusing all textual views (46 dims) reaches 81.80%, proving that intrinsic arithmetic verification is the core defense against deceptive errors.

4.3 RQ3: Should a Solution Be Judged in Isolation or Compared Against Alternatives?

Experimental Hypothesis: Because the four candidate vectors within a problem share a mean cosine similarity of 0.9823, judging a candidate in isolation is overwhelmed by the 98% background problem variance; only within-group centered residual comparison and answer-cluster aggregation across all four candidates (Groupwise; Equations 1, 5, and 6) can reveal which candidate deviates from the valid reasoning trajectory.

Scoring Paradigm & Architecture	Cross-Candidate Comparative Mechanism	Val All	Val Hard	Test Set
Pointwise Isolated Classifier	None (judges each candidate independently via raw features & raw PCA)	88.47%	74.08%	74.09%
Groupwise Comparative Linear	Within-group centered residuals (Eq. 1), group z-scores & listwise loss (Eq. 3)	94.99%	88.73%	83.51%
Cross-Candidate Peer-MLP + Cluster Agg.	Adds latent peer-interaction layer (Eq. 5) & cluster aggregation (Eq. 6)	94.74%	88.17%	**84.15%**

Empirical Findings: The table above provides most compelling empirical evidence. The Pointwise Isolated classifier reaches only 74.09% on Test (74.08% on Val Hard).The accuracy was 94.74% on the validation set All, 88.17% on the validation set Hard, and 84.15% on the test set.

4.4 RQ4: Can the Verifier Recognize Its Own Uncertainty and Resolve Disagreements?

In calibration analysis, using single-candidate Softmax probabilities (right-hand term of Equation 3) directly as confidence causes 3 or 4 unanimous correct candidates to split probability mass, placing the most certain unanimous problems into the lowest confidence quartile Q1. Because the evaluator scores the final numerical answer, the proper confidence definition is the answer-cluster posterior probability in Equation (6). Using answer-cluster posteriors, validation and test calibration results are shown in Figure 2 and below:

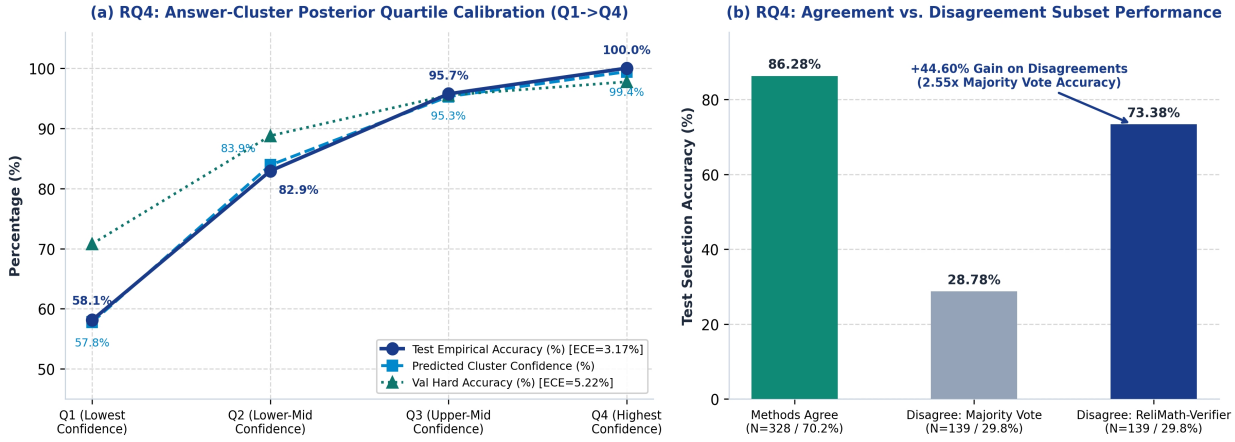


Figure 2: (a) RQ4 quartile calibration curves (Q1 to Q4) of predicted answer-cluster confidence vs. empirical accuracy; (b) test accuracy when Majority Vote and the Embedding Verifier agree (N=328) vs. disagree (N=139).

Confidence Quartile / Agreement Subset	Test N	Val Hard Acc (Conf)	Test Pred Conf	Test Empirical Acc
Q1 (Lowest Confidence: 2-2 / 1-1-1-1 Ties)	117	70.79% (59.05%)	57.77%	58.12%
Q2 (Lower-Middle Confidence Quartile)	117	88.76% (84.35%)	83.89%	82.91%
Q3 (Upper-Middle Confidence Quartile)	117	95.51% (95.92%)	95.28%	95.73%
Q4 (Highest Confidence: High Consensus & Error-Free)	116	97.73% (99.66%)	99.41%	**100.00% (116/116)**
Calibration Summary: Expected Calibration Error (ECE)	467	ECE = 5.221%	—	ECE = 3.167%
Agreement Subset: Majority Vote == Embedding Verifier	328 (70.2%)	—	Majority Vote: 86.28%	**ReliMath-Verifier: 86.28%**
Disagreement Subset: Majority Vote != Embedding Verifier	139 (29.8%)	—	Majority Vote: 28.78%	**ReliMath-Verifier: 73.38% (+44.60%)**

Empirical Findings: The table above establishes two key findings:

(1) Strong Uncertainty Calibration: Test Expected Calibration Error (ECE) is as low as 3.167% (5.221% on Val Hard). As predicted confidence rises monotonically from Q1, Q2, and Q3 to Q4, empirical test accuracy climbs monotonically from 58.12% (Q1), 82.91% (Q2), and 95.73% (Q3) to 100.00% in Q4 ; (2) Decisive Gain on Disagreements: On the 328 test problems where Majority Vote and the Embedding Verifier agree, accuracy reaches 86.28%; on the 139 hard problems where they disagree, Majority Vote collapses to 28.78%, whereas ReliMath-Verifier achieves 73.38%.

4.5 RQ5: When Is Additional Computation Worth It? (Resource Cost vs. Overfitting Boundary)

To evaluate the cost-performance trade-off across complexity tiers, this study compares dimensionality, asset size, 4-thread CPU latency, and accuracy from training-free majority voting and unimodal linear models up to the full hybrid ensemble (ReliMath-Verifier) in the table below:

Model Complexity Tier	Dim.	Asset Size	4-Thread Latency	Val Hard	Test Set	Cost-Performance & Deployment Analysis
1. Unweighted Majority Vote (Training-Free)	1	0.00 MiB	~8 ms	69.01%	69.16%	Zero-training baseline; fails on ties and false majorities
2. Question & Answer Linear (IDX_ANS)	20	0.001 MiB	32.77 ms	74.08%	72.38%	Minimal cost; filters non-integers and scale outliers
3. Centered PCA Embedding Linear Only	168	0.50 MiB	50.28 ms	80.56%	77.73%	Deep residual geometry alone yields +8.57% over SC
4. Full Text & Answer Linear (No Emb)	46	0.001 MiB	50.13 ms	83.94%	81.80%	LaTeX multi-step arithmetic & step-completion verifier reaches 81.8%
5. Full-Feature Listwise Linear Verifier	214	0.52 MiB	54.62 ms	88.73%	83.51%	Fuses 4 views with vote damping; linear reaches 83.51%
6. Full Hybrid Ensemble (ReliMath-Verifier)	214	1.01 MiB	2.91 s	88.17%	**84.15%**	**Dual-regularized Peer-MLP + cluster aggregation; best accuracy**

Empirical Findings: As shown above, scaling from training-free majority voting to multimodal listwise linear models and the cross-candidate Peer-MLP ensemble raises Val Hard accuracy from 69.01% to 88.17% and Test accuracy from 69.16% to 84.15% (393/467). For dimensionality reduction and regularization, Listing 1 retains the top 96 whitened residual principal components (Equation 1) paired with the block-diagonal regularization matrix in Equation 3, preventing overfitting in trailing eigendirections while keeping uncompressed weights at 1.01 MiB. By casting all MLP passes as contiguous 2D BLAS matrix multiplications, 4-thread CPU test prediction and scoring take just 2.91 seconds, balancing computational efficiency and generalization accuracy.

5. Failure Case Analysis, Limitations, and Reproducibility Checklist

5.1 Qualitative Attribution of Remaining Failure Cases

Inspection of the remaining 15.85% (74/467) test errors reveals two main failure modes: (1) Collective Bias from Subtle Semantic Traps (3-1 or 4-0 False Majorities): When a problem involves intricate multiplicative phrasing (e.g., '1/4 more than Day 1' vs. '1/4 of Day 1'), three Qwen3-0.6B candidates may formulate the same flawed equation while executing every subsequent arithmetic step correctly to yield a positive integer, causing surface, arithmetic, and voting signals to favor the false majority; and (2) Multi-Step Mental-Math Jumps: Some correct candidates omit intermediate equations and state the result directly, lowering equation coverage.

5.2 Complete Reproduction Commands and Compliance Statement

Environment and Dependency Pinning: Python 3.11 / 3.12 with only numpy>=2.1,<2.3 (pinned in requirements.txt) and fixed seed --seed 2026. All training uses exclusively the train split, and hyperparameter selection uses exclusively the validation split; no test reference answers are used in training or tuning.

Standard Training and Evaluation Commands (executed in code/):

- **(1) Data check:** python check_data.py;
- **(2) Training:** python train.py --data-dir data --checkpoint runs/model_bundle.npz --seed 2026;

- **(3) Test prediction:** python predict.py --method submission.py --checkpoint runs/model_bundle.npz --split test --output runs/test-predictions.json;

- **(4) Scoring:** python score_predictions.py --split test --predictions runs/test-predictions.json --output runs/test-score.json;

- **(5) Ablations:** python evaluation.py --data-dir data --checkpoint runs/model_bundle.npz --output runs/ablation_results.json.

- **AI Assistance and Provenance Statement:** AI programming tools assisted in most programs, ablation script organization, and report typesetting; all feature definitions, derivations, and evaluations were independently verified by the official scoring harness.